SEMANTIC SEARCH

OG PROJECT

The Breakdown

- **The Problem:** Traditional video platforms only search text metadata like titles and tags, making it impossible to find specific, unlabelled moments buried inside a video without manual scrubbing.
- **The Solution:** I built ChronoSearch, an AI-powered semantic video search engine. It uses computer vision, specifically Google's SigLIP model, to "watch" and understand videos frame-by-frame. It automatically extracts frames, converts them into 1152-dimensional vector embeddings, and indexes them in LanceDB on a serverless Modal backend.
- **The Impact:** Users can search for highly specific visual concepts—like typing "Thanos" as shown in the deep search UI of `image_3a666a.jpg`—and instantly jump to the exact second that visual appears on screen, transforming how we navigate video content.

The Conversational Pitch

When an interviewer says, "Tell me about this project," you want to sound natural, not like you are reading a README file. Here is how you can deliver it:

"I built ChronoSearch to help people find exact moments hidden inside long videos. Usually, you can only search a video's title or tags, but I wanted a way to search the actual video content.

To do this, I used Google's AI to essentially 'watch' the video frame by frame. This means a user can type in something they want to see—like 'a dog running'—and the AI will instantly find the exact second that happens. I also set up a custom video player so the video starts playing right from that timestamp without any buffering delay."

Tech Stack & Architecture

Core Languages & Frameworks:

- **Backend:** Python, FastAPI, Modal (Serverless), LanceDB, OpenCV.
- **AI Model:** Google's SigLIP (Vision-Language Model).
- **Frontend:** React, Vite, Tailwind CSS.

Why these tools? (Your Defenses):

- **Python & FastAPI:** Python is the industry standard for computer vision pipelines. FastAPI was chosen for its high-speed, asynchronous routing, which is essential when handling simultaneous video streams and database queries.
- **SigLIP (over standard CLIP):** Given your focus on Vision-Language Models, you can confidently explain that SigLIP offers superior semantic understanding and zero-shot performance compared to older models like OpenAI's CLIP, making it perfect for matching complex text queries to visual frames.
- **Modal (Serverless GPU):** While you can code and test on your MacBook Air M4, deploying a heavy VLM requires serious hardware. Modal allows you to spin up T4 GPUs dynamically on the cloud, ensuring high performance without paying for an always-on server.
- **LanceDB:** Standard relational databases cannot handle high-dimensional arrays efficiently. LanceDB is purpose-built for fast cosine similarity searches on vector embeddings, keeping search times under a millisecond.
- **OpenCV:** The most robust tool for precise frame extraction and resizing, allowing you to implement the smart 1 FPS sampling logic reliably.

System Flow (The Data Pipeline)

Interviewers love when you can trace a piece of data from the user's click all the way to the database and back. Memorize this flow:

Phase 1: Ingestion (Upload)

1. **Client:** The user uploads a video via the React frontend.
2. **Extraction:** The FastAPI backend uses OpenCV to sample the video at 1 Frame Per Second.
3. **Vectorization:** Modal orchestrates a T4 GPU to pass these frames through the SigLIP model, generating a 1152-dimensional semantic vector for each frame.
4. **Storage:** These vectors, along with metadata (timestamps), are indexed into LanceDB on a persistent cloud volume.

Phase 2: Retrieval (Deep Search)

1. **Query:** The user types a visual concept (e.g., "Thanos").
2. **Embedding:** The backend runs that text query through SigLIP to generate a text vector.
3. **Matching:** LanceDB performs a cosine similarity search between the text vector and the stored frame vectors to find the highest matches.
4. **Streaming:** The frontend requests the specific timestamp of the match, and FastAPI serves it instantly using an HTTP 206 Partial Content response.

3. The "Interview Gold" (Challenges & Optimizations)

Biggest Challenge: The System Was Too Slow

- **The Problem:** At first, trying to process every single frame of a video crashed the system because it was way too much data for the AI to handle all at once.
- **The Fix:** I fixed this in two ways. First, I changed the code to only extract one frame per second (1 FPS), which removed all the unnecessary duplicate images. Second, I moved the heavy AI work to Modal, so it could use powerful cloud GPUs on-demand instead of relying on a single slow server.

Key Algorithms Used: AI Matching

- **SigLIP Model:** I used Google's SigLIP because it is incredibly good at connecting words to images. It takes the video frames and turns them into a list of numbers (called vectors).
- **Cosine Similarity:** To actually search the video, I used Cosine Similarity inside LanceDB. When a user types a search, it turns their text into numbers and compares it to the video frame numbers. The closest math match is the exact scene they are looking for.

What you would do differently today: Make it Faster

- **Use C++ for Speed:** I built the video frame extraction part in Python because it was fast to develop. But if I rebuilt it today, I would write that specific part in C++. It handles memory much better and would make processing large video uploads a lot faster.
- **Test Newer AI:** AI moves fast. Today, I would test newer vision models to see if they are better at understanding blurry frames or fast-moving action scenes compared to the model I originally used.

4. Code Refresher (The Core Files)

You do not need to memorize the entire codebase, Amaliya. You only need to understand the files that do the heavy lifting. Before your placement interview, open these three files and remind yourself how they work:

- `backend/extract.py`
- **What to review here:** Look at how you used OpenCV to break the video down. Make sure you can explain the exact lines of code that force it to grab only one frame per second instead of every single frame.
- `backend/search.py` **(and** `index.py`**)**
- **What to review here:** This is the brain of the project. Review how the code takes the images and passes them to the AI model. Remind yourself how you wrote the LanceDB search query to find the closest math match.
- `backend/api.py`
- **What to review here:** Focus on the custom video streaming route. Review how you handle the fast-forwarding logic so the video jumps straight to the correct timestamp without lagging.

`Extract .py`

How the Code Works (Simple Flow)

- **Folder Setup**: First, the code checks if a folder exists to store the images. If not, it creates a new one.
- **Opening the Video:** It opens the video file and finds out its original speed (usually around 24 or 30 frames per second).
- **The Math**: It calculates how many frames it needs to skip so it only grabs exactly two frames for every second of the video.
- **Shrinking the Image**: When it grabs a target frame, it shrinks it down to 640 pixels wide. It calculates the new height automatically so the image doesn't look stretched or distorted.
- **Saving with Timestamps:** It saves each resized image into the folder, using the exact second it appeared in the video as the file name (for example, frame_1.50.jpg).

Potential Interview Questions

Interviewers will want to know why you made specific technical choices. You can easily connect these questions back to core concepts in digital image processing, like temporal sampling and pixel interpolation.

- **"Why did you choose exactly 2 frames per second?"**
    - *How to answer:* Explain that 2 FPS is the sweet spot for temporal sampling. It captures enough visual movement without creating a massive amount of duplicate images that would slow down the AI later.
- **"What is happening under the hood when you use `cv2.resize`?"**
    - *How to answer:* This is a great place to bring up digital image processing. Mention that OpenCV uses interpolation algorithms to safely shrink the image matrix. Shrinking it to 640 pixels drastically reduces the compute load for the SigLIP model compared to processing full-size images.
- **"What happens to the server memory if a user uploads a massive 4K video?"**

    - *How to answer:* Explain that reading the video frame-by-frame and resizing it *before* saving prevents the server's hard drive from filling up with massive, high-resolution image files.
- **"Why use the timestamp as the filename instead of a simple counter like 1, 2, 3?"**
    - *How to answer:* You can explain that saving the exact time directly in the filename makes the later search process much faster. When the AI finds a match, the system immediately knows what timestamp to send to the video player.

Search .py

How the Code Works (Simple Flow)

- **Waking up the AI:** First, the code loads Google's SigLIP model into the server's memory. It checks if there is a powerful GPU available to make things run faster; if not, it uses the standard CPU.
- **Connecting to the Database:** It connects to LanceDB, which is the specialized database where all the previously extracted video frame numbers (vectors) are stored.
- **Translating Words into Numbers:** When a user searches for a word (like "Thanos"), the AI translates that exact word into a long list of numbers (a text vector).
- **Scaling the Numbers (Normalization):** The code adjusts these numbers so they are on the exact same scale as the video frame numbers. This ensures the math comparison later is fair and accurate.
- **Finding the Match:** It asks the database to compare the text vector against millions of video frame vectors. It uses **"Cosine Similarity"** to find the frames that point in the exact same semantic direction as the text.
- **Formatting for the User:** The database returns a "distance" score (where 0 means a perfect match). The code subtracts this from 1 to turn it into a standard percentage score (like 95% Match) and sends the top 10 highest-scoring timestamps back to the screen.

Potential Interview Questions

Interviewers will zero in on this file because it shows your understanding of how AI models integrate with databases.

- **"Why do you use `torch.no_grad()` before processing the text?"**
    - *How to answer:* Explain that you are only *using* the model for inference (searching), not training it. Using `torch.no_grad()` tells PyTorch to stop tracking memory

for backpropagation, which saves a massive amount of server RAM and makes the search instantly faster.

- **"Why is 'Normalization' a critical step here?"**
  - *How to answer:* Explain that because you are using Cosine Similarity, the length or magnitude of the vector doesn't matter; only the direction does. Normalizing forces all vectors to have a length of 1, ensuring the math strictly compares the meaning of the image, not the size of the data.
- **"Why did you use LanceDB instead of a normal SQL database like PostgreSQL?"**
  - *How to answer:* Explain that standard databases are built for text and numbers, not high-dimensional arrays. LanceDB is specifically built to store and search vector embeddings extremely quickly without slowing down as the library grows.
- **"Can you explain your score logic (`score_row['_distance']`)?"**
  - *How to answer:* Explain that LanceDB's cosine metric returns a *distance*. In distance, a smaller number is better (0 is an identical match). But users prefer percentages where a *higher* number is better. Subtracting the distance from 1 flips the logic so it can be displayed as an intuitive percentage on the frontend.

INDEX .PY

How the Code Works (Simple Flow)

- **Waking up the AI**: Just like the search file, it loads the SigLIP model into memory, prioritizing the powerful GPU to make the processing fast.
- **Setting up the Database:** It connects to LanceDB and prepares two separate tables (like two separate spreadsheets). One table is for the video frames (Deep Search), and the other is for the video titles and tags (Global Search).
- **Filing the Text (Metadata):** It takes the video's title and tags, squishes them together into one sentence, and asks the AI to turn that text into a vector (a list of numbers). It saves this into the Global Search table.
- **The Cleanup (Crucial Step):** Before it looks at the images, it deletes any old frames belonging to this specific video ID. This ensures that if a user re-uploads or updates a video, old "ghost" frames don't stay stuck in the database.
- **Reading the Images:** It opens the folder of extracted frames and sorts them in perfect chronological order.
- **Vectorizing and Batching:** It loops through every image, has the AI turn the picture into numbers, normalizes the math, and holds it in a temporary list called a "buffer."
- **Bulk Saving**: Instead of saving the frames to the database one by one, it dumps the entire buffer into the database all at once to save time.

Potential Interview Questions

Interviewers will love this file because it shows you understand database management, batch processing, and edge cases (like handling re-uploads).

- **"Why did you use two separate tables for metadata and frames instead of putting everything in one?"**

  How to answer: Explain that this is for speed and scaling. Global search only needs to scan one row per video to find a title match. Deep search needs to scan thousands of rows per video to find a frame. Keeping them separate means a simple title search doesn't have to sift through millions of image vectors.

- **"Why do you use a `buffer` list to save the frames?"**

  How to answer: This is called "Batch Processing." Tell them that writing data to a database one row at a time creates a huge bottleneck. By saving all the vectors in memory first and pushing them to LanceDB in one single command (`tbl_frames.add(buffer)`), the database insertion happens almost instantly.

- **"Can you explain the 'ghost results' problem and how your code fixes it?"**

  How to answer: Explain that in the real world, users update or overwrite files. If a user replaces a 5-minute video with a 2-minute version under the same ID, the database would still hold the vectors for minutes 3, 4, and 5. If a search hits one of those old frames, the video player would crash trying to jump to a timestamp that no longer exists. Explicitly deleting old data first prevents this completely.

API .PY

How the Code Works (Simple Flow)

- **The Traffic Cop:** This file uses FastAPI to create "routes" (like `/upload` or `/search`). Every time a user clicks a button on the website, it hits one of these routes.
- **Uploading in the Background (`/upload`):** When a user uploads a video, this code saves the actual file to the cloud's hard drive and adds the title to the database. Most importantly, it uses a command called `.spawn()` to start the heavy AI processing (extracting frames and making vectors) in the *background*.
- **Video Streaming (`/stream`):** When the frontend needs to play a video, this route finds the file on the hard drive and sends it back using `FileResponse`. Because you are using FastAPI, this automatically supports the special protocol (Range Requests) needed to jump instantly to a specific timestamp without buffering.
- **The Repair Button (`/reindex`):** If a video's search gets broken, a user can click "Repair." The code checks if the file exists and restarts the AI indexer.
- **Spam Protection:** Inside the repair route, there is a "Cooldown Timer." If someone clicks the repair button, it locks the button for 5 minutes. This prevents people from clicking it 100 times in a row and crashing the system.

Potential Interview Questions

Interviewers look closely at API files to see if you understand web architecture, user experience, and server security.

- **"In the `/upload` route, why did you use `.spawn()` to run the AI process?"**
  - *How to answer:* Explain that AI processing takes time. If you ran it normally, the user's browser would freeze on a loading screen for minutes while the video processes, and the browser might even timeout. Using `.spawn()` kicks the heavy lifting to the background, so the server can immediately tell the user, "Upload successful!" while the AI works quietly behind the scenes.
- **"How exactly does your streaming route allow zero-latency seeking to a specific timestamp?"**
  - *How to answer:* Mention that returning a `FileResponse` in FastAPI automatically handles "HTTP 206 Partial Content." This means when the AI says "Go to 01:45," the frontend only asks the server for the video bytes starting at 01:45. The server doesn't force the browser to download the first 1 minute and 44 seconds, which makes the playback instant.
- **"I see you added a cooldown timer in the `/reindex` route. Why is that important?"**
  - *How to answer:* Explain this as "Rate Limiting" or "Spam Protection." Since re-indexing a video spins up a cloud GPU, it is an expensive and heavy task. If a malicious user clicked that endpoint 1,000 times, it would crash the server or rack up a massive cloud bill. The 5-minute cooldown prevents abuse.